# Assessment-4

**CFD analysis of steady incompressible flow through a T-junction using OpenFOAM**

05.08.2026

Ajay Pokhrel
ajaypokhrel981@gmail.com

# Table of Contents


| | | |
|---|---|---|
| 1 | Table of Contents | 2 |
| 2 | Objective | 3 |
| 3 | Geometry Description | 4 |
| 4 | Methodology | 5-20 |
| 5 | Results | 21-22 |
| 6 | Conclusion | 23 |

# Objective

- To perform a CFD simulation of steady incompressible flow through a T-junction pipe using OpenFOAM.
- To generate a computational mesh from the STL geometry using *snappyHexMesh* and solve the flow using the simpleFoam solver.
- To analyse the velocity and pressure distribution within the pipe

# Geometry Description

- The computational domain consists of a three-dimensional **T-junction pipe** represented by an STL geometry. The pipe has one inlet and two outlets, allowing the incoming flow to split into two branches. The remaining surfaces are treated as solid walls. The geometry is imported into OpenFOAM and used for mesh generation through the snappyHexMesh utility

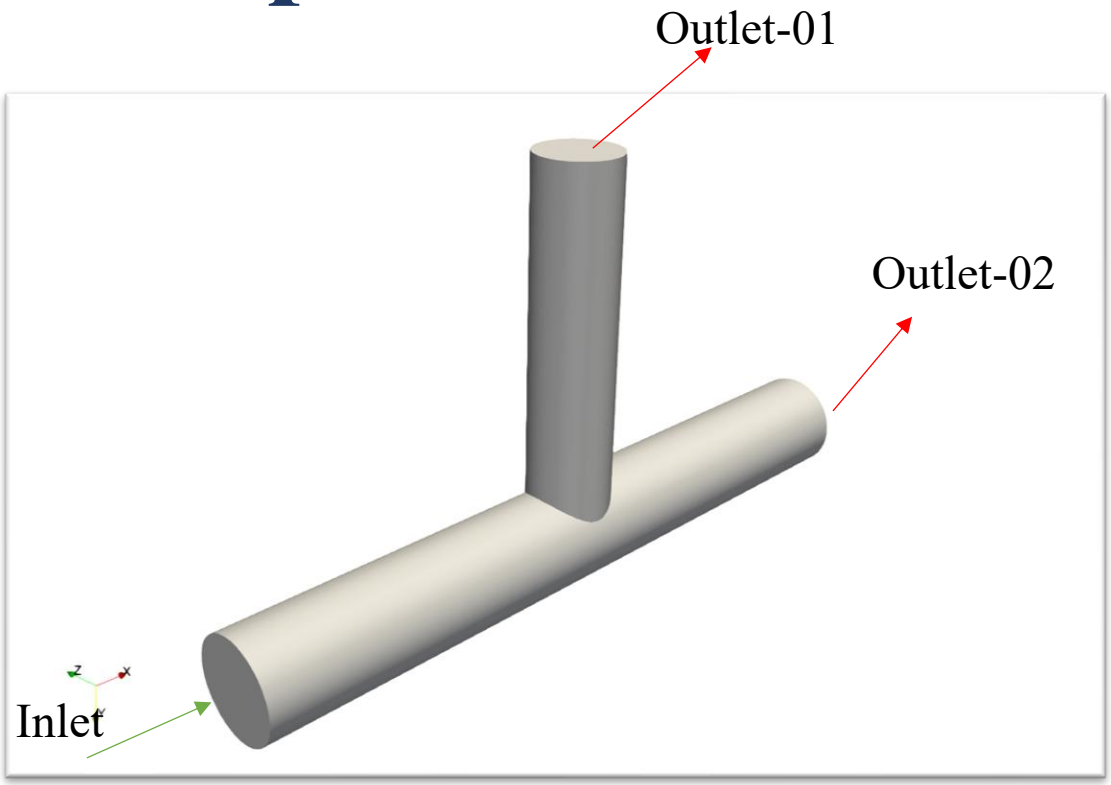



**Figure 1.** *Three-dimensional T-junction pipe geometry used for the CFD simulation*

# Methodology

1. Case Preparation
2. Mesh Generation
3. Simulation setup
4. Solver Settings

# 1. Case preparation

- The *motorBike* tutorial case provided with OpenFOAM was used as the base case for this simulation. A copy of the tutorial was created and renamed as pipe_flow. The motorcycle geometry was replaced with the T-junction pipe STL file (pipe.stl), which was placed inside the constant/triSurface directory. The existing case structure was then modified to suit the pipe flow simulation

  - cp -r $FOAM_TUTORIALS/incompressible/simpleFoam/motorBike pipe_flow
  - cd pipe_flow
  - mkdir constant/triSurface
  - cp /path/to/pipe.stl constant/triSurface/

# 2. Mesh Generation

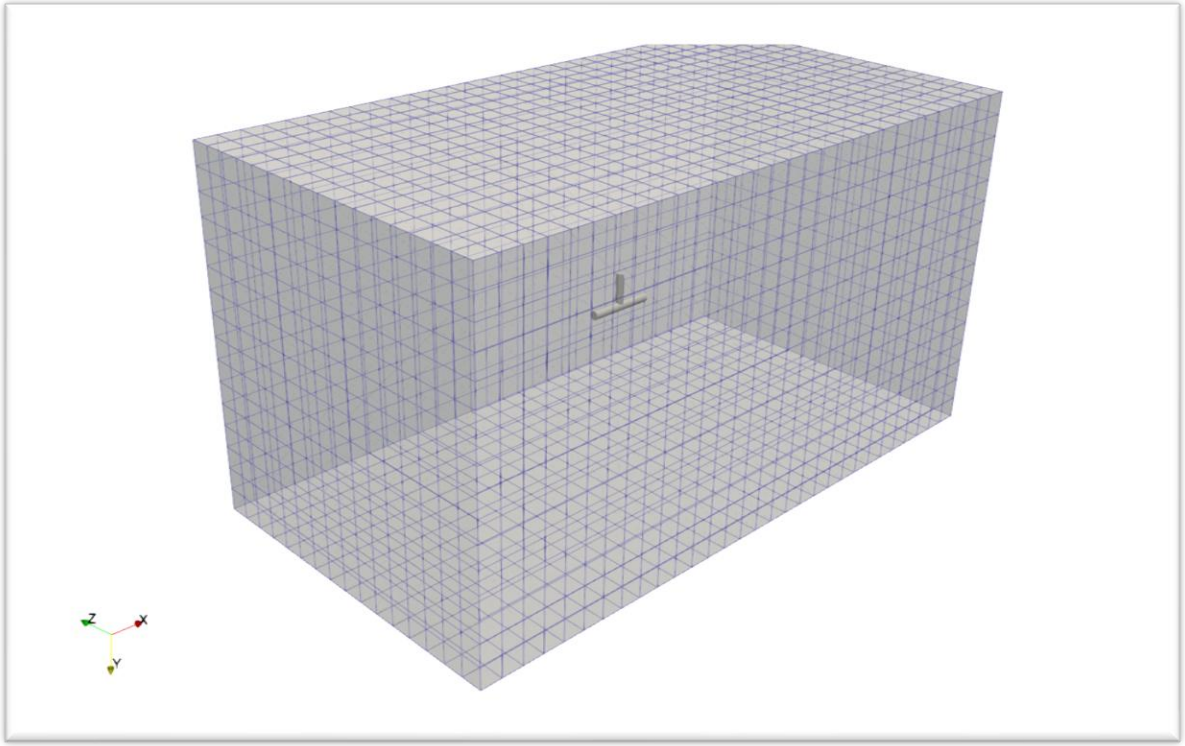
- **Background Mesh generation**

  The background mesh was generated using the blockMesh utility. A single hexahedral block enclosing the entire T-junction geometry was defined in the blockMeshDict file. The mesh was discretized into **30 × 16 × 16** structured hexahedral cells with **uniform grading (simpleGrading (1 1 1))**. This background mesh served as the initial computational mesh for the subsequent snappyHexMesh refinement process.

**Table 1:** BlockMesh result

| parameter | value |
|---|---|
| Mesh utility | blockMesh |
| Block Type | Hexahedral |
| No. of blocks | 1 |
| Cell Divisions | 30 × 16 × 16 |
| Total background cells | 7680 |
| Grading | simpleGrading(1 1 1) |

# 2. Mesh Generation



```
Creating points with scale (1 1 1)
    Block 0 cell size :
        i : 0.15 .. 0.15
        j : 0.15 .. 0.15
        k : 0.15 .. 0.15
```

**Figure 2:** *Background mesh generated using the blockMesh utility.*

# 2. Mesh Generation

- **Surface Feature Extraction:** After generating the background mesh, the surfaceFeatureExtract utility was used to identify and extract the sharp feature edges from the STL geometry. The extracted edges were saved as an .eMesh file and later used by snappyHexMesh to preserve the geometric features during mesh generation

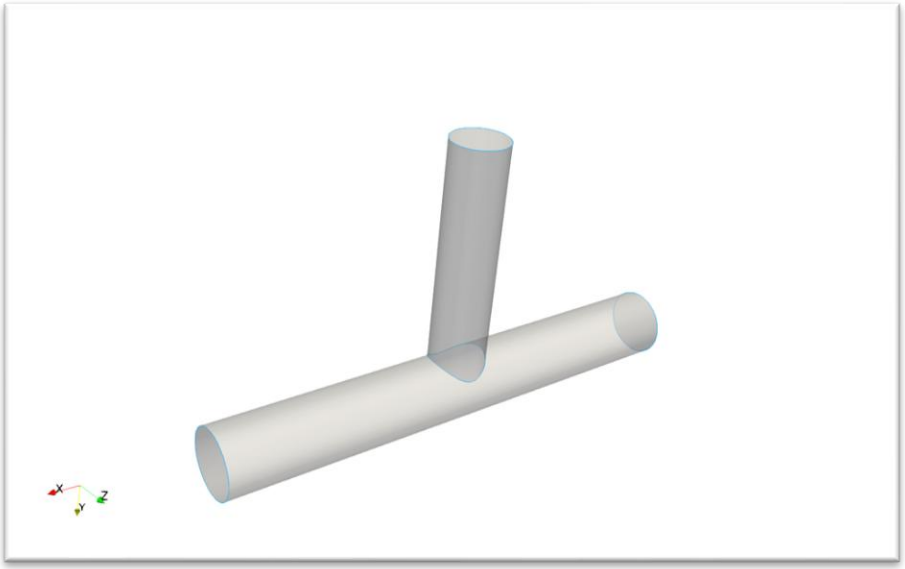


**Fig 3:** *surface feature extraction of pipe.*

# 2. Mesh Generation

- **Final Mesh Generation(snappyHexMesh):** Since it is internal flow, we have to define *locationInMesh* inside of pipe.stl

```
locationInMesh (0.10001 0.021011 0.025222);
```

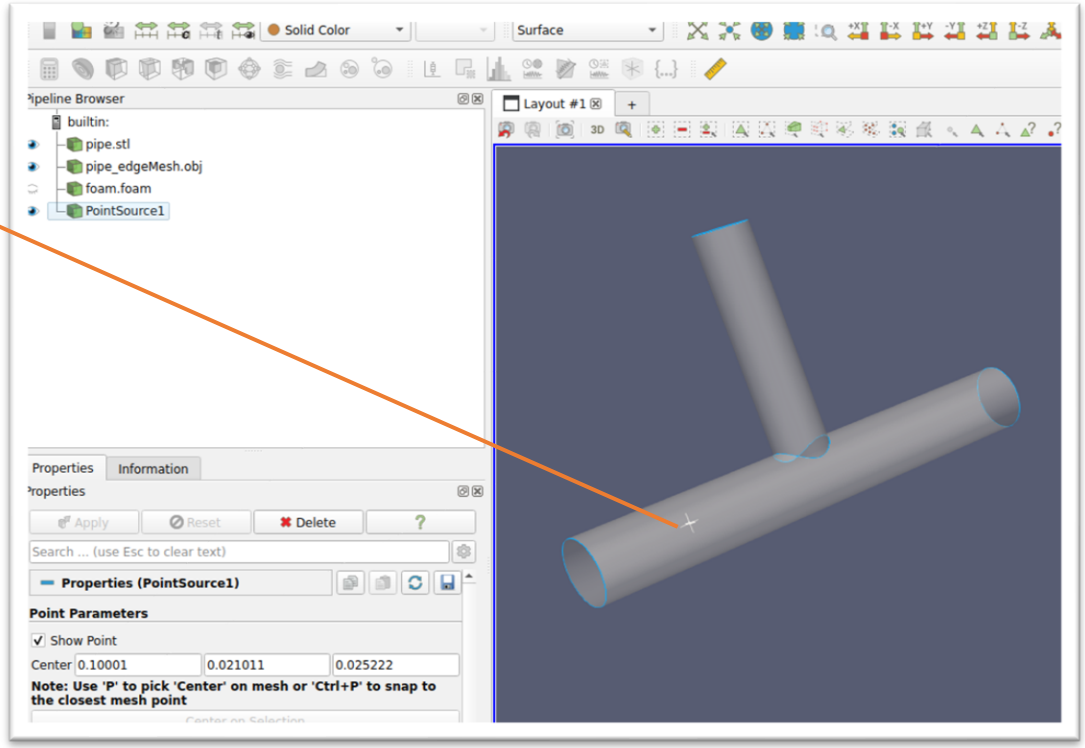



**Figure 4:** *Point inside the pipe.*

# 2. Mesh Generation

- **Final Mesh Generation(snappyHexMesh):**

```
// Which of the steps to run
castellatedMesh true;
snap            true;
addLayers       false;//true;
```

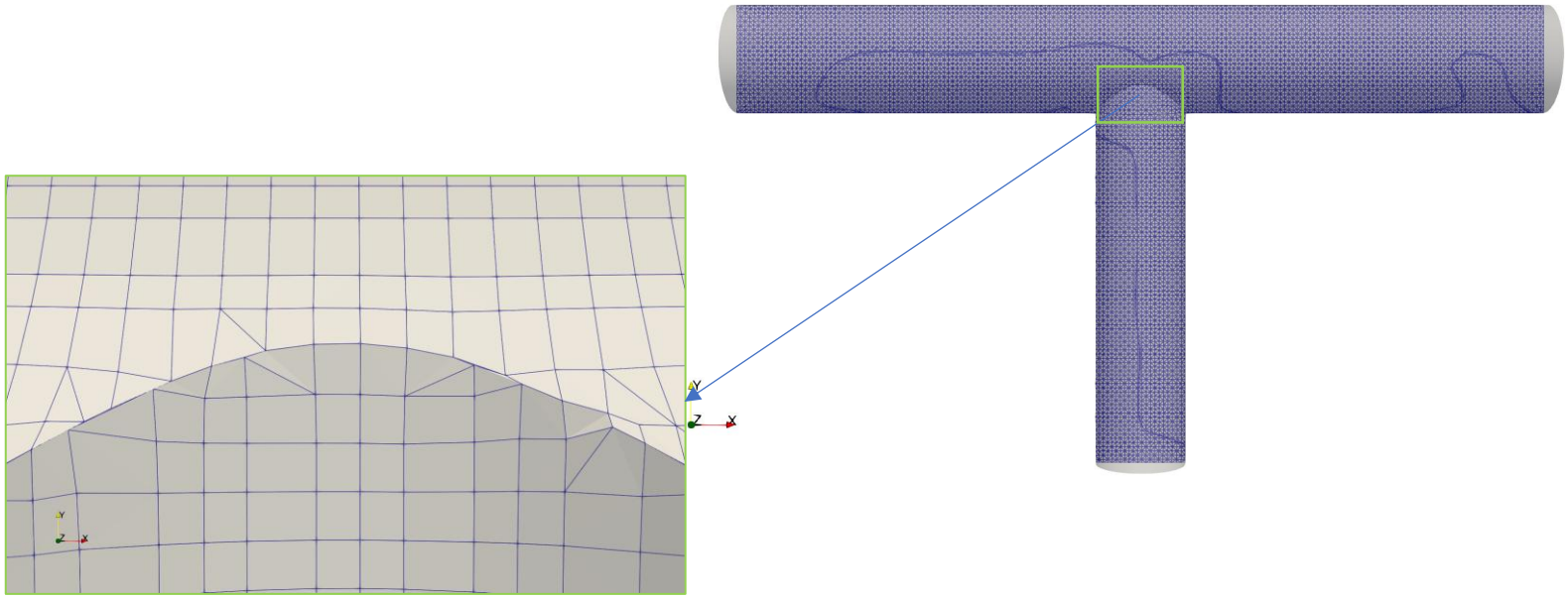


**Figure 5:** *Meshing using snappyHexmesh without adding boundary layer.*

# 2. Mesh Generation

- **Final Mesh Generation(snappyHexMesh):**

```
// Which of the steps to run
castellatedMesh false;//true
snap            false;//true
addLayers       true;
```

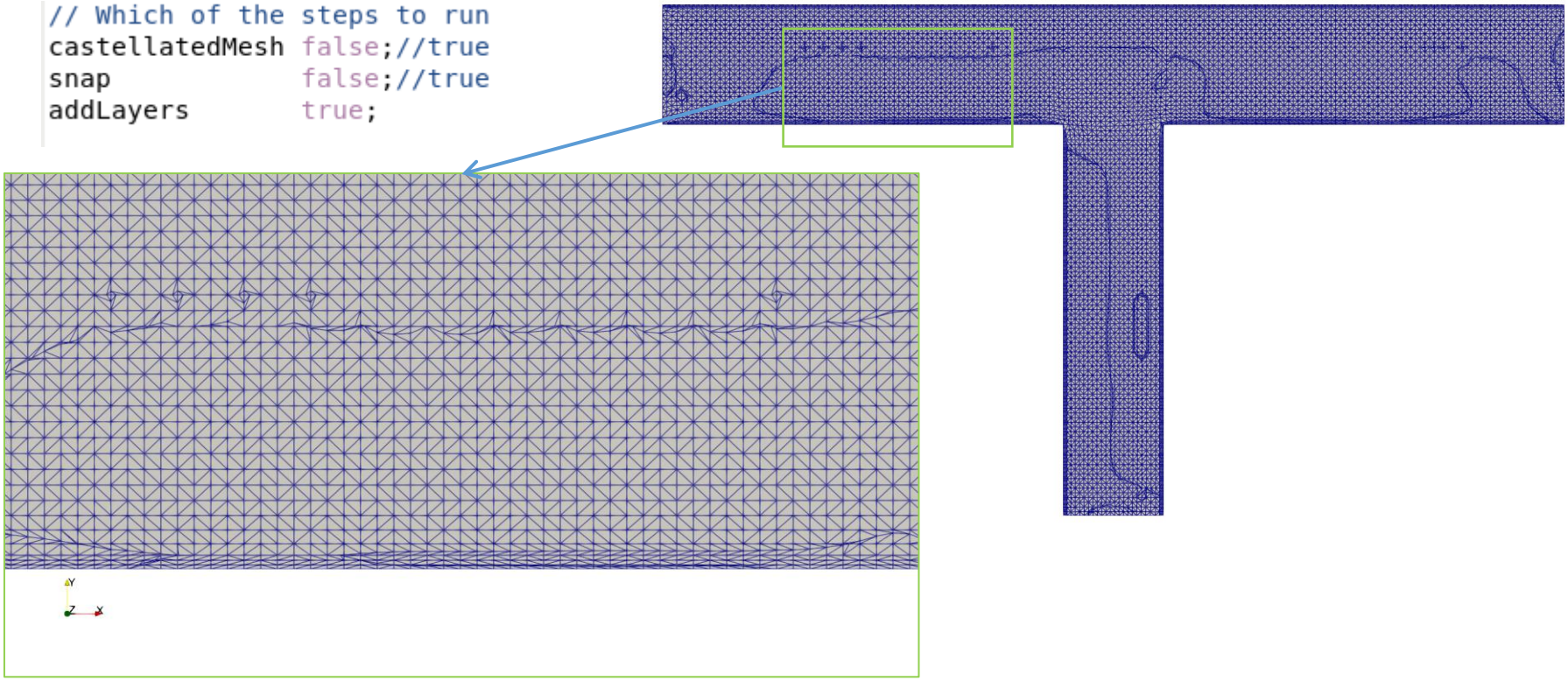


**Figure 5:** *Meshing using snappyHexmesh with adding boundary layer.*

# 2. Mesh Generation

- **Check Mesh:**

```
Mesh stats
    points:           189459
    faces:            555769
    internal faces:   535705
    cells:            183310
    faces per cell:   5.95425
    boundary patches: 4
    point zones:      1
    face zones:       0
    cell zones:       0

Overall number of cells of each type:
    hexahedra:     174252
    prisms:        8658
    wedges:        0
    pyramids:      0
    tet wedges:    0
    tetrahedra:    0
    polyhedra:     400
    Breakdown of polyhedra by number of faces:
        faces   number of cells
            4   22
            5   66
            6   30
            7   194
            8   80
            9   4
           10   4
```

```
Checking geometry...
    Overall domain bounding box (-1.13524e-06 -0.2 -0.00499471) (0.45 0.055 0.0549947)
    Mesh has 3 geometric (non-empty/wedge) directions (1 1 1)
    Mesh has 3 solution (non-empty) directions (1 1 1)
    Boundary openness (-1.69211e-18 1.77906e-17 3.21592e-17) OK.
    Max cell openness = 3.48994e-16 OK.
    Max aspect ratio = 7.45115 OK.
    Minimum face area = 1.84564e-07. Maximum face area = 1.06176e-05.  Face area magnitudes OK.
    Min volume = 1.143e-10. Max volume = 1.81289e-08.  Total volume = 0.00165779.  Cell volumes OK.
    Mesh non-orthogonality Max: 63.5633 average: 3.69491
    Non-orthogonality check OK.
    Face pyramids OK.
    Max skewness = 2.51797 OK.
    Coupled point location match (average 0) OK.

Mesh OK.

End
```

```
Checking patch topology for multiply connected surfaces...
                Patch    Faces   Points  Surface topology
           pipe_walls    18120    18266  ok (non-closed singly connected)
                inlet      686      725  ok (non-closed singly connected)
             outlet-01      532      548  ok (non-closed singly connected)
             outlet-02      726      745  ok (non-closed singly connected)
                  ".*"    20064    20070      ok (closed singly connected)
```

# 3. Simulation Setup

- **Transport Properties:**

```
FoamFile
{
    version     2.0;
    format      ascii;
    class       dictionary;
    object      transportProperties;
}
// * * * * * * * * * * * * * * * * * * * *

transportModel  Newtonian;

nu              1.5e-05;
```

Air kinematic viscosity

- **Turbulence model**

```
FoamFile
{
    version     2.0;
    format      ascii;
    class       dictionary;
    object      turbulenceProperties;
}
// * * * * * * * * * * * * * * * * * * * *

simulationType      RAS;

RAS
{
    RASModel        kOmegaSST;

    turbulence      on;

    printCoeffs     on;
}
```

# 3. Simulation Setup

- **Boundary condition in O folder:**

**U file**

```
    object      U;
}
// * * * * * * * * * * * * * * * * * * * * * * * * *

#include        "include/initialConditions"

dimensions      [0 1 -1 0 0 0 0];

internalField   uniform $flowVelocity;

boundaryField
{
    #includeEtc "caseDicts/setConstraintTypes"

    #include "include/fixedInlet"

    inlet
    {
    type  fixedValue;
    value $internalField;
    }

    outlet-01
    {
        type            inletOutlet;
        inletValue      uniform (0 0 0);
        value           $internalField;
    }

    outlet-02
    {
        type            inletOutlet;
        inletValue      uniform (0 0 0);
        value           $internalField;
    }

    pipe_walls
    {
        type            fixedValue;
        value           $internalField;
    }
```

```
flowVelocity        (1 0 0);
pressure            0;
turbulentKE         0.001;
turbulentOmega      2.916;
```

```
inlet
{
    type   fixedValue;
    value $internalField;
}
```

**p file**

```
    object      p;
}
// * * * * * * * * * * * * * * * * * * * * * * * * *

#include        "include/initialConditions"

dimensions      [0 2 -2 0 0 0 0];

internalField   uniform $pressure;

boundaryField
{
    #includeEtc "caseDicts/setConstraintTypes"

    inlet
    {
        type            zeroGradient;
    }

    outlet-01
    {
        type            fixedValue;
        value           $internalField;
    }

    outlet-02
    {
        type            fixedValue;
        value           $internalField;
    }

    pipe_walls
    {
        type            zeroGradient;
    }

}
```

# 3. Simulation Setup

- **Boundary condition in O folder:**

**k file**

```
    object      k;
}
// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * *

#include        "include/initialConditions"

dimensions      [0 2 -2 0 0 0 0];

internalField   uniform $turbulentKE;

boundaryField
{
    #includeEtc "caseDicts/setConstraintTypes"

    inlet
    {
    type  fixedValue;
    value $internalField;
    }

    outlet-01
    {
        type            inletOutlet;
        inletValue      $internalField;
        value           $internalField;
    }

    outlet-02
    {
        type            inletOutlet;
        inletValue      $internalField;
        value           $internalField;
    }

    pipe_walls
    {
        type            kqRWallFunction;
        value           $internalField;
    }

}
```

**omega file**

```
    object      omega;
}
// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * *

#include        "include/initialConditions"

dimensions      [0 0 -1 0 0 0 0];

internalField   uniform $turbulentOmega;

boundaryField
{
    #includeEtc "caseDicts/setConstraintTypes"

    inlet
    {
        type  fixedValue;
        value $internalField;
    }

    outlet-01
    {
        type            inletOutlet;
        inletValue      $internalField;
        value           $internalField;
    }

    outlet-02
    {
        type            inletOutlet;
        inletValue      $internalField;
        value           $internalField;
    }

    pipe_walls
    {
        type            omegaWallFunction;
        value           $internalField;
    }

}
```

**nut file**

```
    object      nut;
}
// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * *

dimensions      [0 2 -1 0 0 0 0];

internalField   uniform 0;

boundaryField
{
    #includeEtc "caseDicts/setConstraintTypes"

    inlet
    {
        type            calculated;
        value           uniform 0;
    }

    outlet-01
    {
        type            calculated;
        value           uniform 0;
    }

    outlet-02
    {
        type            calculated;
        value           uniform 0;
    }

    pipe_walls
    {
        type            nutkWallFunction;
        value           uniform 0;
    }

}
```

# 3. Simulation Setup

**Table 2:** *Boundary Conditions Used in the Simulation*

| Boundary patch | Velocity(U) | Pressure(p) | k | w | nut |
|---|---|---|---|---|---|
| Inlet | ( 1 0 0 ) m/s | Zero Gradient | 0.001 $m^2/s^2$ | 2.916 $s^{-1}$ | Calculated |
| Outlet -01 | InletOutlet | 0 | InletOutlet | InletOutlet | Calculated |
| Outlet-02 | InletOutlet | 0 | InletOutlet | InletOutlet | Calculated |
| Pipe_walls | No-slip | Zero Gradient | *kqRWallFunction* | omegaWallFunction | nutkWallFunction |

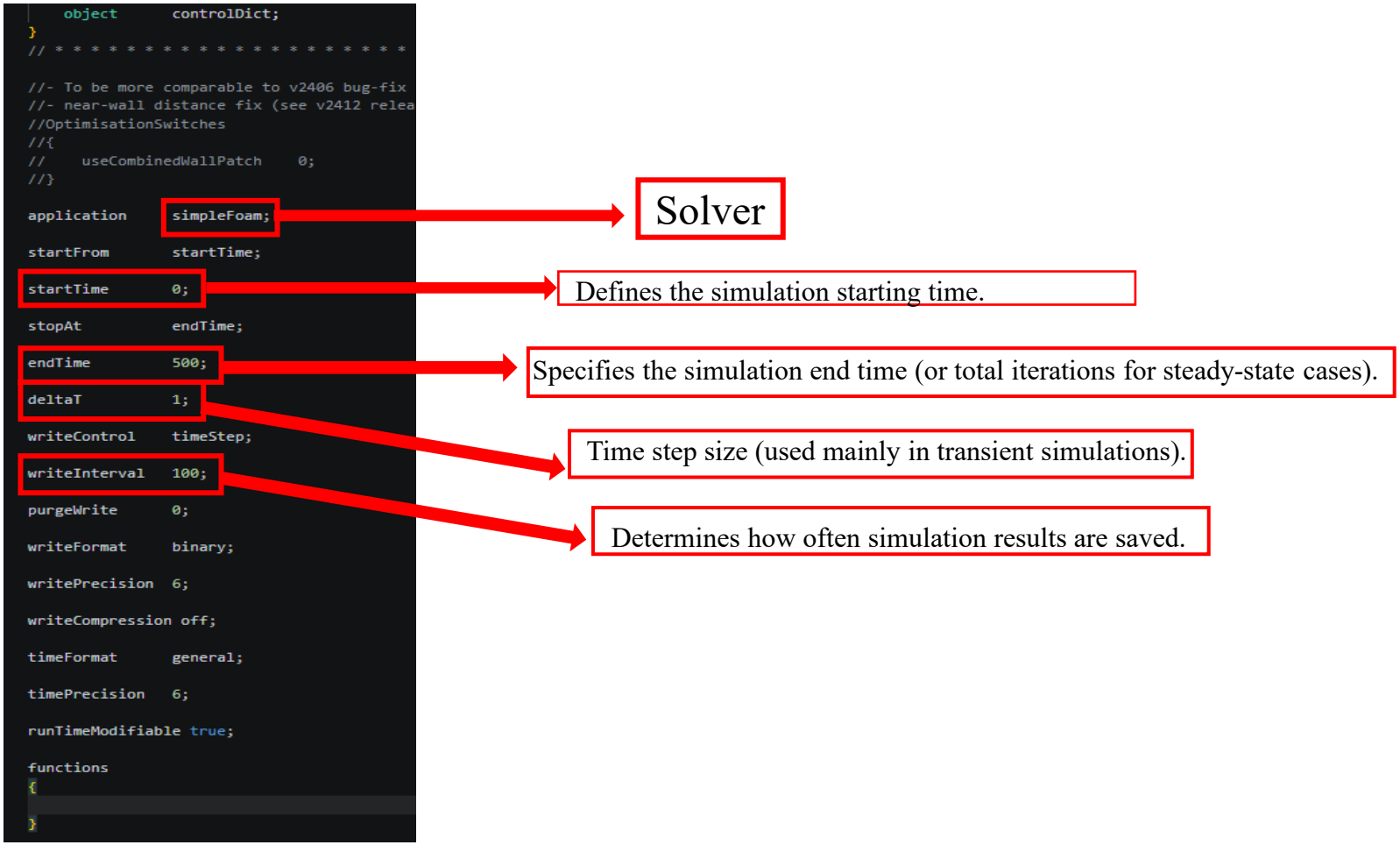
# 4. Solver Settings

The solver settings were defined in the *controlDict*, *fvSchemes*, and *fvSolution* files. These files control the simulation time, numerical discretization schemes, pressure–velocity coupling, and equation solvers

| Setting | File | Description |
|---|---|---|
| Solver | **simpleFoam** | Steady-state incompressible flow solver |
| Simulation Control | **controlDict** | Controls iterations and output |
| Numerical Schemes | **fvSchemes** | Defines discretization schemes |
| Linear Solvers | **fvSolution** | Defines equation solvers and SIMPLE settings |

# 4. Solver Settings

- **ControlDict :** The controlDict file controls the overall simulation, including the solver name, simulation start and end time, write interval, and output settings



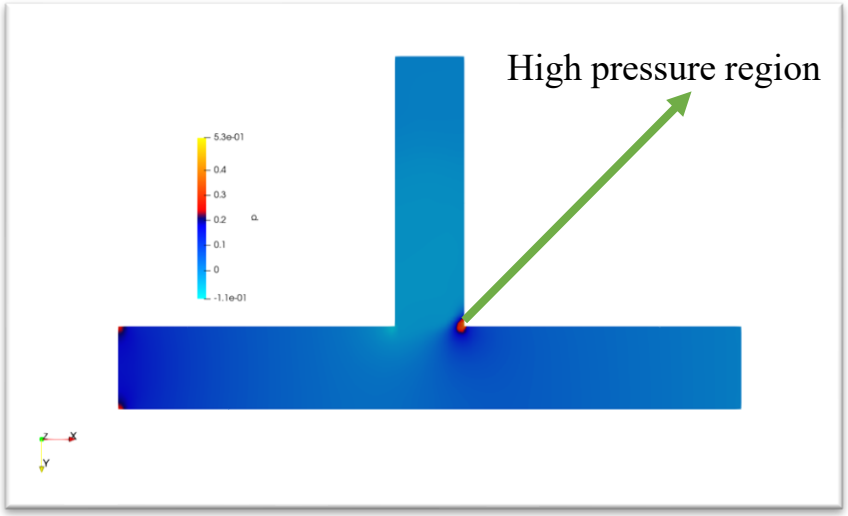
# 4. Solver Settings

- **fvScheme :** The *fvSchemes* file defines the numerical discretization schemes used for gradients, divergence, laplacian, interpolation, and time derivatives in the finite volume method.

- **fvSolution:** The *fvSolution* file specifies the linear solvers, solver tolerances, relaxation factors, and the SIMPLE algorithm settings used during the simulation.

# Results

- **Pressure Distribution**

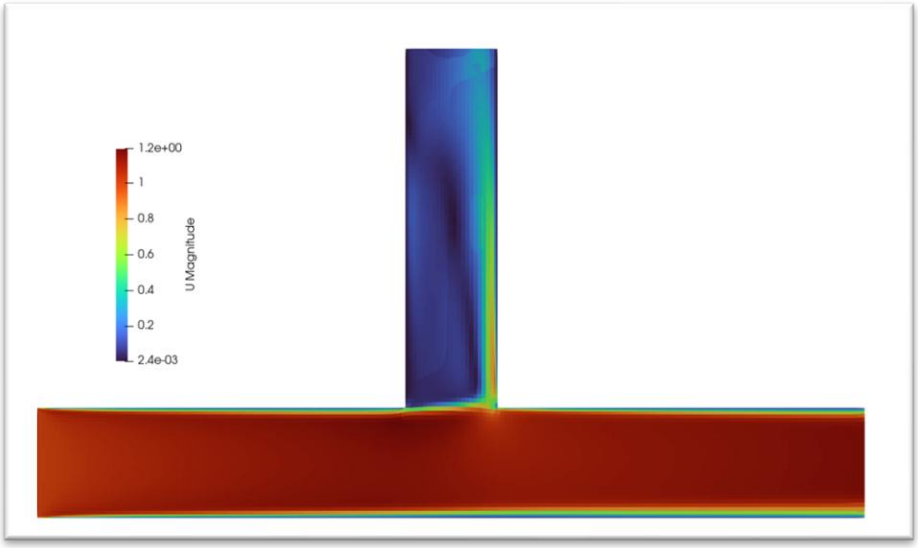



- Pressure is highest near the inlet and the T-junction.
- Pressure decreases gradually toward both outlets.
- The pressure drop is caused by flow division and frictional losses.
- Both outlets maintain approximately **0 Pa gauge pressure**, as specified by the boundary condition

**Figure 7:** Pressure contour of the T-junction at 500 iterations.

# Results

- **Velocity Distribution**




- The maximum velocity is observed in the main horizontal pipe.
- The flow splits at the T-junction and enters the vertical branch.
- Velocity decreases in the vertical branch due to flow division.
- Low velocity is observed near the pipe walls because of the **no-slip** boundary condition.

**Figure 8:** Velocity magnitude contour of the T-junction at 500 iterations.

# Conclusion

- This case study provided practical experience with the complete OpenFOAM simulation workflow.
- The study included geometry preparation, mesh generation, boundary condition setup, solver configuration, and post-processing in ParaView.
- Pressure and velocity contours were successfully generated and analyzed to understand the flow behavior in a T-junction pipe.
- The assessment improved understanding of OpenFOAM case structure, simulation setup, and result visualization.
- Overall, this hands-on case study strengthened the fundamental knowledge required for performing CFD simulations using OpenFOAM.